Week 1, Day 2

DNS, HTTP and HTTPS: How the Web Really Works

Goal of today: understand everything that happens between typing a website name and seeing the page, and know which parts an attacker can see, change or abuse.

Every abbreviation is written with its full form where it is used.

1. Quiz review: what to remember

- **DNS (Domain Name System) is the phonebook of the internet,** not of one user. It maps names to IP (Internet Protocol) addresses.
- **Give the full sequence in interviews:** browser cache, then recursive resolver, then root server, then TLD (Top-Level Domain) server, then authoritative name server. Then TCP (Transmission Control Protocol) connection, TLS (Transport Layer Security) handshake, and the HTTP (HyperText Transfer Protocol) request.
- **401 (Unauthorized)** = not logged in. **403 (Forbidden)** = logged in but not allowed. **404 (Not Found)** = resource does not exist.
- **HTTP is stateless.** The server does not remember you, so the browser sends a cookie or session token with every request. Stealing it means session hijacking.
- **TLS gives three things:** confidentiality, integrity and authentication. An attacker on the same network still sees IP addresses, the domain name, timing and traffic size, but not the content.
- **HTTPS protects data only in transit.** SQL (Structured Query Language) injection and XSS (Cross-Site Scripting) happen inside the encrypted tunnel.
- **Compression is not encryption.** Compression (for example gzip, GNU zip) has no key and anyone can reverse it. Encryption needs a secret key.

2. DNS (Domain Name System)

Computers use IP (Internet Protocol) addresses, but people remember names. DNS turns a name such as neverssl.com into an IP address.

```
You type: neverssl.com
 1. Browser cache, then operating system cache
 2. Recursive resolver (your ISP or 8.8.8.8 / 1.1.1.1)
 3. Root server          -> "ask the .com servers"
 4. TLD server (.com)    -> "ask the name server of neverssl.com"
 5. Authoritative server -> returns the IP address
 6. Answer is cached for the TTL (Time To Live)
 7. Browser opens TCP connection -> TLS handshake -> HTTP request
(ISP = Internet Service Provider)
```

DNS normally uses **UDP (User Datagram Protocol) port 53** because queries are small. It uses **TCP port 53** for large answers and zone transfers.

Record	Full form / meaning	Why a security person cares
A	Address record: domain to IPv4 (Internet Protocol version 4)	Finds the server to scan
AAAA	Domain to IPv6 (Internet Protocol version 6)	Often forgotten and less protected

Record	Full form / meaning	Why a security person cares
CNAME	Canonical Name: an alias to another name	Dangling aliases lead to subdomain takeover
MX	Mail Exchange: mail servers of the domain	Shows email setup for phishing recon
NS	Name Server: who answers for the domain	Targets for zone transfer tests
TXT	Text: free text, used for SPF (Sender Policy Framework) and verification	Leaks which services a company uses
PTR	Pointer: IP back to a name (reverse lookup)	Identifies what an IP belongs to

DNS attacks to know

- **DNS spoofing / cache poisoning:** feeding a resolver a fake answer so users reach the attacker's server.
- **DNS tunneling:** hiding data inside DNS queries to get past firewalls.
- **Zone transfer (AXFR, Authoritative Zone Transfer) misconfiguration:** a server hands its whole record list to anyone who asks.
- **Subdomain enumeration:** guessing names like admin, dev, test to find forgotten and weaker systems. A core recon skill.
- **Subdomain takeover:** a CNAME points to a service that no longer exists, and an attacker claims it.

Tools: when to use which

Tool	Use it when	Example
nslookup (Name Server Lookup)	A quick, simple lookup	nslookup neverssl.com
dig (Domain Information Groper)	You want detailed output or a specific record type	dig google.com MX
whois	Early recon: who registered the domain and when	whois neverssl.com
Wireshark filter dns	Watching real DNS queries and answers	Filter: dns

3. HTTP (HyperText Transfer Protocol) in depth

HTTP is **stateless**: each request is independent. Websites add memory using **cookies** and **session tokens**, which the browser sends with every request.

Method	Purpose	Security note
GET	Fetch data	Data is in the URL (Uniform Resource Locator), so it appears in logs and history
POST	Send data, such as a form	Data is in the body, not the URL
PUT	Replace or upload a resource	Dangerous if left open (file upload attacks)
DELETE	Remove a resource	Needs strict access control
HEAD	Headers only, no body	Quietly checks what a server is
OPTIONS	Ask which methods are allowed	Reveals what the server permits

Range	Meaning	Examples
1xx	Informational	101 Switching Protocols
2xx	Success	200 OK, 201 Created
3xx	Redirect	301 Moved Permanently, 302 Found
4xx	Client error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
5xx	Server error	500 Internal Server Error, 503 Service Unavailable

Security tips: a 500 error after odd input suggests you broke something, which hints at a vulnerability. A 403 on /admin tells you the page exists even though you cannot open it.

Reading your Day 1 capture

- **User-Agent** reveals browser and operating system. **Server: Apache/2.4.68** reveals the web server and exact version (information disclosure).
- **Content-Encoding: gzip** means the body is compressed, not encrypted.
- **If-Modified-Since / ETag (Entity Tag)** are cache checks.
- **Referer** shows the previous page (the spelling error is in the official standard).

Important security headers

- **Set-Cookie** with flags: **HttpOnly** (JavaScript cannot read it) and **Secure** (sent only over HTTPS, HyperText Transfer Protocol Secure).
- **Strict-Transport-Security** (HSTS, HTTP Strict Transport Security): the browser must always use HTTPS.
- **Content-Security-Policy** (CSP): limits which scripts can run, helping against XSS (Cross-Site Scripting).

4. HTTPS and TLS (Transport Layer Security)

HTTPS is HTTP inside a TLS tunnel. TLS gives **confidentiality** (encryption), **integrity** (no undetected changes) and **authentication** (the server is genuine, proved by a certificate).

```
TLS handshake (simplified)
 1. Client Hello      : supported TLS versions and ciphers
 2. Server Hello      : chosen cipher + digital certificate
 3. Certificate check : signed by a trusted CA (Certificate Authority)?
                        correct domain? not expired?
 4. Key exchange      : both sides agree on a shared secret key
 5. Encrypted data    : symmetric encryption from now on
```

- **Asymmetric encryption** (public and private key pair): slow, used only to agree on the key safely.
- **Symmetric encryption** (one shared key): fast, used for the actual data.
- **What an attacker still sees:** IP addresses, domain name, timing and size of traffic. **Not visible:** passwords, cookies, page content.
- **Common TLS findings in tests:** expired or self-signed certificates, old versions (SSL, Secure Sockets Layer 3.0 or TLS 1.0), weak ciphers, no redirect from HTTP to HTTPS.

- **Remember:** HTTPS protects data in transit. It does not stop SQL (Structured Query Language) injection, XSS or weak passwords.

5. Hands-on lab for today (about 90 minutes)

Part A: Terminal (25 min)

```
dig neverssl.com A
dig google.com MX
dig google.com NS
dig -x 8.8.8.8          # reverse lookup (PTR record)
nslookup neverssl.com
whois neverssl.com
```

Compare the results and note the TTL (Time To Live) value in the answers.

Part B: Wireshark (25 min)

- Start a capture, run a dig command, filter with `dns`, and find your query and its response.
- Open an HTTPS site and filter with `tls`. Find the **Client Hello** and spot the domain name inside it.
- Compare with your Day 1 plain HTTP capture: readable versus encrypted.

Part C: Browser developer tools (10 min)

- Press F12, open the **Network** tab, load a page, click one request, and read its headers, status code and cookies.

Part D: TryHackMe (30 min)

- In the Pre Security path, complete the **DNS in Detail** and **HTTP in Detail** rooms.

6. Self-test (answer without looking)

- List the steps from typing a URL to seeing the page, in order.
- Name four DNS record types and what each stores.
- What is the difference between 401, 403 and 404?
- Why is HTTP stateless, and how do cookies fix that?
- What are the three guarantees of TLS?
- What can an attacker still see on HTTPS traffic?
- Compression versus encryption: what is the key difference?

7. Interview-ready answers

- **What happens when you type a URL?** The browser checks its DNS cache, asks the resolver, which queries root, TLD and authoritative servers to get the IP. The browser opens a TCP connection (three-way handshake), completes the TLS handshake if HTTPS, sends the HTTP request, receives the response and renders the page.
- **HTTP vs HTTPS?** HTTPS is HTTP over TLS, which adds encryption, integrity and server authentication.
- **Is HTTPS enough to be secure?** No. It protects data in transit only. Application flaws like SQL injection and XSS still work inside the tunnel.

- **Encoding vs compression vs encryption?** Encoding changes format and is reversible by anyone. Compression reduces size and is reversible by anyone. Encryption hides data and needs a secret key.

8. Day 2 checklist

- [ ] Ran dig, nslookup and whois and noted the TTL
- [ ] Found a DNS query and response in Wireshark
- [ ] Found a TLS Client Hello in Wireshark
- [ ] Inspected headers in the browser Network tab
- [ ] Finished the TryHackMe DNS in Detail and HTTP in Detail rooms
- [ ] Wrote a half-page note of what I learned

Tomorrow (Day 3): Linux fundamentals, the file system, permissions, users and the command line, starting with Kali Linux.